

causal-learn Documentation

Source: <https://causal-learn.readthedocs.io/en/latest/>

Welcome to causal-learn's documentation!

causal-learn is a Python translation and extension of the Tetrad java code. It offers the implementations of up-to-date causal discovery methods as well as simple and intuitive APIs.

Note

This project is under active development. For source code, please kindly refer to our GitHub Repository .

Contents

- Getting started Installation Running examples Benchmarks Contributors Citation
- Installation
- Running examples
- Benchmarks
- Contributors
- Citation
- Search methods Constraint-based causal discovery methods PC FCI CD-NOD Score-based causal discovery methods GES with the BIC score or generalized score DGES: GES in the Presence of Deterministic Relations Exact Search Causal discovery methods based on constrained functional causal models LiNGAM-based Methods Post-nonlinear causal models Additive noise models Hidden causal representation learning Generalized Independence Noise (GIN) condition-based method Permutation-based causal discovery methods GRaSP BOSS Granger causality Linear granger causality
- Constraint-based causal discovery methods PC FCI CD-NOD
- PC
- FCI

- CD-NOD
 - Score-based causal discovery methods GES with the BIC score or generalized score DGES: GES in the Presence of Deterministic Relations Exact Search
 - GES with the BIC score or generalized score
 - DGES: GES in the Presence of Deterministic Relations
 - Exact Search
 - Causal discovery methods based on constrained functional causal models LiNGAM-based Methods Post-nonlinear causal models Additive noise models
 - LiNGAM-based Methods
 - Post-nonlinear causal models
 - Additive noise models
 - Hidden causal representation learning Generalized Independence Noise (GIN) condition-based method
 - Generalized Independence Noise (GIN) condition-based method
 - Permutation-based causal discovery methods GRaSP BOSS
 - GRaSP
 - BOSS
 - Granger causality Linear granger causality
 - Linear granger causality
 - (Conditional) independence tests Fisher-z test Missing-value Fisher-z test Chi-Square test Kernel-based conditional independence (KCI) test and independence test G-Square test
 - Fisher-z test
 - Missing-value Fisher-z test
 - Chi-Square test
 - Kernel-based conditional independence (KCI) test and independence test
 - G-Square test
 - Score functions BIC score BDeu score Generalized score with cross validation Generalized score with marginal likelihood
 - BIC score
 - BDeu score
 - Generalized score with cross validation
 - Generalized score with marginal likelihood
 - Utilities Graph operations Evaluations Datasets Graph Visualization
 - Graph operations
 - Evaluations
 - Datasets
 - Graph Visualization
-

Getting started

Installation

Requirements

- python 3 (≥ 3.7)

python 3 (≥ 3.7)

- numpy

numpy

- networkx

networkx

- pandas

pandas

- scipy

scipy

- scikit-learn

scikit-learn

- statsmodels

statsmodels

- pydot

pydot

(For visualization)

- matplotlib

matplotlib

- graphviz

graphviz

- pygraphviz (might not support the most recent Mac)

pygraphviz (might not support the most recent Mac)

Install via PyPI

To use causal-learn, we could install it using pip :

```
(.venv) $ pip install causal-learn
```

Install from source

For development version, please kindly refer to our GitHub Repository .

Running examples

For search methods in causal discovery, there are various running examples in the ‘tests’ directory in our GitHub Repository , such as TestPC.py and TestGES.py.

For the implemented modules, such as (conditional) independent test methods, we provide unit tests for the convenience of developing your own methods.

Benchmarks

For the convenience of our community, CMU-CLeaR group maintains a list of benchmark datasets including real-world scenarios and various learning tasks. Please refer to the following links:

- <https://github.com/cmu-phil/example-causal-datasets>
(maintained by Joseph Ramsey)

<https://github.com/cmu-phil/example-causal-datasets> (maintained by Joseph Ramsey)

- <https://www.cmu.edu/dietrich/causality/causal-learn>

<https://www.cmu.edu/dietrich/causality/causal-learn>

Please feel free to let us know if you have any recommendation regarding causal datasets with high-quality. We are grateful for any effort that benefits the development of causality community.

Contributors

Team Leaders : Kun Zhang, Joseph Ramsey, Mingming Gong, Ruichu Cai, Shohei Shimizu, Peter Spirtes, Clark Glymour

Coordinators : Yujia Zheng, Biwei Huang, Wei Chen

Developers :

- All community contributors

All community contributors

- Wei Chen, Biwei Huang, Yuequn Liu, Zhiyi Huang, Feng Xie, Haoyue Dai, Xiaokai Huang, Yiwen Qiu: PC , FCI , GES , GIN , and graph operations .

Wei Chen, Biwei Huang, Yuequn Liu, Zhiyi Huang, Feng Xie, Haoyue Dai, Xiaokai Huang, Yiwen Qiu: PC , FCI , GES , GIN , and graph operations .

- Mingming Gong, Erdun Gao, Aoqi Zuo, Oliver Schacht: PNL , ANM , Granger causality , KCI , and some other tests (e.g., RCIT, FastKCI).

Mingming Gong, Erdun Gao, Aoqi Zuo, Oliver Schacht: PNL , ANM , Granger causality , KCI , and some other tests (e.g., RCIT, FastKCI).

- Shohei Shimizu, Takashi Nicholas Maeda, Takashi Ikeuchi: LiNGAM-based methods .

Shohei Shimizu, Takashi Nicholas Maeda, Takashi Ikeuchi: LiNGAM-based methods .

- Madelyn Glymour: several helpers.

Madelyn Glymour: several helpers.

- Ruibo Tu: Missing-value/test-wise deletion PC .

Ruibo Tu: Missing-value/test-wise deletion PC .

- Wai-Yin Lam: PC .

Wai-Yin Lam: PC .

- Biwei Huang: CD-NOD .

Biwei Huang: CD-NOD .

- Ignavier Ng, Yujia Zheng: Exact search .

Ignavier Ng, Yujia Zheng: Exact search .

- Bryan Andrews, Joseph Ramsey: GRaSP , BOSS .

Bryan Andrews, Joseph Ramsey: GRaSP , BOSS .

- Joseph Ramsey, Wei Chen, Zhiyi Huang: Evaluations .

Joseph Ramsey, Wei Chen, Zhiyi Huang: Evaluations .

- Longkang Li, Haoyue Dai: DGES .

Longkang Li, Haoyue Dai: DGES .

Quality control : Yewen Fan, Zhiyi Huang, Haoyue Dai, Yujia Zheng, Ignavier Ng, Xiangchen Song

Citation

Please cite as:

```
@article{zheng2024causal,  
  title={Causal-learn: Causal discovery in python},  
  author={Zheng, Yujia and Huang, Biwei and Chen, Wei and Ramsey,  
Joseph and Gong, Mingming and Cai, Ruichu and Shimizu, Shohei and  
Spirtes, Peter and Zhang, Kun},  
  journal={Journal of Machine Learning Research},  
  volume={25},  
  number={60},  
  pages={1- -8},  
  year={2024}  
}
```

Welcome to causal-learn's documentation!

causal-learn is a Python translation and extension of the Tetrad java code. It offers the implementations of up-to-date causal discovery methods as well as simple and intuitive APIs.

Note

This project is under active development. For source code, please kindly refer to our GitHub Repository .

Contents

- Getting started Installation Running examples Benchmarks Contributors Citation
- Installation
- Running examples
- Benchmarks
- Contributors
- Citation
- Search methods Constraint-based causal discovery methods PC FCI CD-NOD Score-based causal discovery methods GES with the BIC score or generalized score DGES: GES in the Presence of Deterministic Relations Exact Search Causal discovery methods based on constrained functional causal models LiNGAM-based Methods Post-nonlinear causal models Additive noise models Hidden causal representation learning Generalized Independence Noise (GIN) condition-based method Permutation-based causal discovery methods GRaSP BOSS Granger causality Linear granger causality
- Constraint-based causal discovery methods PC FCI CD-NOD
- PC
- FCI
- CD-NOD
- Score-based causal discovery methods GES with the BIC score or generalized score DGES: GES in the Presence of Deterministic Relations Exact Search
- GES with the BIC score or generalized score
- DGES: GES in the Presence of Deterministic Relations
- Exact Search
- Causal discovery methods based on constrained functional causal models LiNGAM-based Methods Post-nonlinear causal models Additive noise models
- LiNGAM-based Methods
- Post-nonlinear causal models
- Additive noise models
- Hidden causal representation learning Generalized Independence Noise (GIN) condition-based method
- Generalized Independence Noise (GIN) condition-based method
- Permutation-based causal discovery methods GRaSP BOSS
- GRaSP
- BOSS
- Granger causality Linear granger causality
- Linear granger causality
- (Conditional) independence tests Fisher-z test Missing-value Fisher-z test Chi-Square test Kernel-based conditional independence (KCI) test and independence test G-Square test
- Fisher-z test
- Missing-value Fisher-z test
- Chi-Square test

- Kernel-based conditional independence (KCI) test and independence test
 - G-Square test
 - Score functions BIC score BDeu score Generalized score with cross validation Generalized score with marginal likelihood
 - BIC score
 - BDeu score
 - Generalized score with cross validation
 - Generalized score with marginal likelihood
 - Utilities Graph operations Evaluations Datasets Graph Visualization
 - Graph operations
 - Evaluations
 - Datasets
 - Graph Visualization
-

Search methods

In this section, we would like to introduce search methods for causal discovery in causal-learn.

Contents:

- Constraint-based causal discovery methods PC Algorithm
Introduction Usage Advanced Usages Parameters Returns FCI
Algorithm Introduction Usage Parameters Returns CD-NOD
Algorithm Introduction Usage Parameters Returns
- PC Algorithm Introduction Usage Advanced Usages
Parameters Returns
- Algorithm Introduction
- Usage
- Advanced Usages
- Parameters
- Returns
- FCI Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- CD-NOD Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- Score-based causal discovery methods GES with the BIC score
or generalized score Algorithm Introduction Usage Parameters
Returns DGES: GES in the Presence of Deterministic Relations
Algorithm Introduction Usage Parameters Returns Exact
Search Algorithm Introduction Usage Parameters Returns

- GES with the BIC score or generalized score Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- DGES: GES in the Presence of Deterministic Relations Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- Exact Search Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- Causal discovery methods based on constrained functional causal models LiNGAM-based Methods ICA-based LiNGAM DirectLiNGAM VAR-LiNGAM RCD CAM-UV Post-nonlinear causal models Algorithm Introduction Usage Parameters Returns Additive noise models Algorithm Introduction Usage Parameters Returns
- LiNGAM-based Methods ICA-based LiNGAM DirectLiNGAM VAR-LiNGAM RCD CAM-UV
- ICA-based LiNGAM
- DirectLiNGAM
- VAR-LiNGAM
- RCD
- CAM-UV
- Post-nonlinear causal models Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- Additive noise models Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- Hidden causal representation learning Generalized Independence Noise (GIN) condition-based method Algorithm Introduction Usage Parameters Returns
- Generalized Independence Noise (GIN) condition-based method Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns

- Permutation-based causal discovery methods GRaSP Algorithm Introduction Usage Parameters Returns BOSS Algorithm Introduction Usage Parameters Returns
 - GRaSP Algorithm Introduction Usage Parameters Returns
 - Algorithm Introduction
 - Usage
 - Parameters
 - Returns
 - BOSS Algorithm Introduction Usage Parameters Returns
 - Algorithm Introduction
 - Usage
 - Parameters
 - Returns
 - Granger causality Linear granger causality Algorithm Introduction Usage Parameters Returns
 - Linear granger causality Algorithm Introduction Usage Parameters Returns
 - Algorithm Introduction
 - Usage
 - Parameters
 - Returns
-

Constraint-based causal discovery methods

In this section, we would like to introduce constraint-based causal discovery methods, including PC [1] , FCI [2] , and CD-NOD [3] .

Contents:

- PC Algorithm Introduction Usage Advanced Usages Parameters Returns
- Algorithm Introduction
- Usage
- Advanced Usages
- Parameters
- Returns
- FCI Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- CD-NOD Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters

- Returns

Spirtes, P., Glymour, C. N., Scheines, R., & Heckerman, D. (2000). Causation, prediction, and search. MIT press.

Spirtes, P., Meek, C., & Richardson, T. (1995, August). Causal inference in the presence of latent variables and selection bias. In Proceedings of the Eleventh conference on Uncertainty in artificial intelligence (pp. 499-506).

Huang, B., Zhang, K., Zhang, J., Ramsey, J. D., Sanchez-Romero, R., Glymour, C., & Schölkopf, B. (2020). Causal Discovery from Heterogeneous/Nonstationary Data. J. Mach. Learn. Res., 21(89), 1-53.

PC

Algorithm Introduction

Perform Peter-Clark (PC [1]) algorithm for causal discovery. We also allowed data sets with missing values, for which testwise-deletion PC is included (choosing 'MV-Fisher_Z' for the test name).

If you would like to use missing-value PC [2] , please set 'mvpc' as True.

Usage

```
from causallearn.search.ConstraintBased.PC import pc # default
parameters cg = pc ( data ) # or customized parameters cg = pc
( data , alpha , indep_test , stable , uc_rule , uc_priority ,
mvpc , correction_name , background_knowledge , verbose ,
show_progress ) # visualization using pydot cg . draw_pydot_graph
() # or save the graph from causallearn.utils.GraphUtils import
GraphUtils pyd = GraphUtils . to_pydot ( cg . G ) pyd . write_png
( 'simple_test.png' ) # visualization using networkx #
cg.to_nx_graph() # cg.draw_nx_graph(skel=False)
```

Visualization using pydot is recommended. If specific label names are needed, please refer to this usage example (e.g., 'cg.draw_pydot_graph(labels=["A", "B", "C"])' or 'GraphUtils.to_pydot(cg.G, labels=["A", "B", "C"])').

Advanced Usages

- If you would like to specify parameters for the (conditional) independence test (if available), you may directly pass the parameters to the pc call. E.g., from `causallearn.search.ConstraintBased.PC` import pc from `causallearn.utils.cit` import kci cg = pc (data , 0.05 , kci , kernelZ = 'Polynomial' , approx = False , est_width = 'median' , ...)

If you would like to specify parameters for the (conditional) independence test (if available), you may directly pass the parameters to the pc call. E.g.,

```
from causallearn.search.ConstraintBased.PC import pc from
causallearn.utils.cit import kci cg = pc ( data , 0.05 , kci ,
kernelZ = 'Polynomial' , approx = False , est_width =
'median' , ... )
```

- If your graph is big and/or your independence test is slow (e.g., KCI), you may want to cache the p-value results to a local checkpoint. Then by reading values from this local checkpoint, no more repeated calculation will be wasted to resume from checkpoint / just finetune some PC parameters. This can be achieved by specifying `cache_path`. E.g., `citest_cache_file = "/my/path/to/citest_cache_dataname_kci.json"` # .json file cg1 = pc (data , 0.05 , kci , `cache_path` = `citest_cache_file`) # after the long run # just finetune `uc_rule`. p-values are reused, and thus cg2 is done in almost no time. cg2 = pc (data , 0.05 , kci , `cache_path` = `citest_cache_file` , `uc_rule` = 1) If `cache_path` does not exist in your local file system, a new one will be created. Otherwise, the cache will be first loaded from the json file to the CIT class and used during the runtime. Note that 1) data hash and parameters hash will first be checked at loading to ensure consistency, and 2) during runtime, the cache will be saved to the local file every 30 seconds.

If your graph is big and/or your independence test is slow (e.g., KCI), you may want to cache the p-value results to a local checkpoint. Then by reading values from this local checkpoint, no more repeated calculation will be wasted to resume from checkpoint / just finetune some PC parameters. This can be achieved by specifying `cache_path`. E.g.,

```
citest_cache_file = "/my/path/to/citest_cache_dataname_kci.json"
# .json file cg1 = pc ( data , 0.05 , kci , cache_path =
citest_cache_file ) # after the long run # just finetune uc_rule.
p-values are reused, and thus cg2 is done in almost no time. cg2 =
pc ( data , 0.05 , kci , cache_path = citest_cache_file , uc_rule
= 1 )
```

If `cache_path` does not exist in your local file system, a new one will be created. Otherwise, the cache will be first loaded from the json file to the CIT class and used during the runtime. Note that 1) data hash and parameters hash will first be checked at loading to ensure consistency, and 2) during runtime, the cache will be saved to the local file every 30 seconds.

- The above advanced usages also apply to other constraint-based methods, e.g., FCI and CDNOD.

The above advanced usages also apply to other constraint-based methods, e.g., FCI and CDNOD.

Parameters

`data` : `numpy.ndarray`, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

`alpha` : desired significance level (float) in (0, 1). Default: 0.05.

- “ `fisherz` ”: Fisher’s Z conditional independence test.

“ `fisherz` ”: Fisher’s Z conditional independence test.

- “ `chisq` ”: Chi-squared conditional independence test.

“ `chisq` ”: Chi-squared conditional independence test.

- “ `gsq` ”: G-squared conditional independence test.

“ `gsq` ”: G-squared conditional independence test.

- “ `kci` ”: kernel-based conditional independence test. (As a kernel method, its complexity is cubic in the sample size, so it might be slow if the same size is not small.)

“ `kci` ”: kernel-based conditional independence test. (As a kernel method, its complexity is cubic in the sample size, so it might be slow if the same size is not small.)

- “ `mv_fisherz` ”: Missing-value Fisher’s Z conditional independence test.

“ `mv_fisherz` ”: Missing-value Fisher’s Z conditional independence test.

(For more efficient nonparametric test, you may try FastKCI and RCIT . Both implementations are still preliminary and there might be some issues.)

stable : run stabilized skeleton discovery [4] if True. Default: True.

- 0: run uc_sepset.

0: run uc_sepset.

- 1: run maxP [3] . Orient an unshielded triple X-Y-Z as a collider with an additional CI test.

1: run maxP [3] . Orient an unshielded triple X-Y-Z as a collider with an additional CI test.

- 2: run definiteMaxP [3] . Orient only the definite colliders in the skeleton and keep track of all the definite non-colliders as well.

2: run definiteMaxP [3] . Orient only the definite colliders in the skeleton and keep track of all the definite non-colliders as well.

- -1: whatever is default in uc_rule.

-1: whatever is default in uc_rule.

- 0: overwrite.

0: overwrite.

- 1: orient bi-directed.

1: orient bi-directed.

- 2: prioritize existing colliders.

2: prioritize existing colliders.

- 3: prioritize stronger colliders.

3: prioritize stronger colliders.

- 4: prioritize stronger* colliders.

4: prioritize stronger* colliders.

mvp : use missing-value PC or not. Default: False.

correction_name . Missing value correction if using missing-value PC. Default: 'MV_Crtn_Fisher_Z'

background_knowledge : class BackgroundKnowledge. Add prior edges according to assigned causal connections. Default: None. For detailed usage, please kindly refer to its usage example .

verbose : True iff verbose output should be printed. Default: False.

show_progress : True iff the algorithm progress should be show in console. Default: True.

Returns

cg : a CausalGraph object, where $cg.G.graph[j,i]=1$ and $cg.G.graph[i,j]=-1$ indicate $i \rightarrow j$; $cg.G.graph[i,j] = cg.G.graph[j,i] = -1$ indicate $i - j$; $cg.G.graph[i,j] = cg.G.graph[j,i] = 1$ indicates $i \leftrightarrow j$.

Spirtes, P., Glymour, C. N., Scheines, R., & Heckerman, D. (2000). Causation, prediction, and search. MIT press.

Tu, R., Zhang, C., Ackermann, P., Mohan, K., Kjellström, H., & Zhang, K. (2019, April). Causal discovery in the presence of missing data. In The 22nd International Conference on Artificial Intelligence and Statistics (pp. 1762-1770). PMLR.

Ramsey, J. (2016). Improving accuracy and scalability of the pc algorithm by maximizing p-value. arXiv preprint arXiv:1610.00378.

Colombo, D., & Maathuis, M. H. (2014). Order-independent constraint-based causal structure learning. J. Mach. Learn. Res., 15(1), 3741-3782.

FCI

Algorithm Introduction

Causal Discovery with Fast Causal Inference (FCI [1]).

Usage

```
from causallearn.search.ConstraintBased.FCI import fci # default
parameters g , edges = fci ( data ) # or customized parameters g ,
edges = fci ( data , independence_test_method , alpha , depth ,
max_path_length , verbose , background_knowledge ,
cache_variables_map ) # visualization from
causallearn.utils.GraphUtils import GraphUtils pdy = GraphUtils .
to_pydot ( g ) pdy . write_png ( 'simple_test.png' )
```

Visualization using pydot is recommended. If specific label names are needed, please refer to this usage example .

Parameters

`dataset` : `numpy.ndarray`, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

- “ `fisherz` ”: Fisher’s Z conditional independence test.

“ `fisherz` ”: Fisher’s Z conditional independence test.

- “ `chisq` ”: Chi-squared conditional independence test.

“ `chisq` ”: Chi-squared conditional independence test.

- “ `gsq` ”: G-squared conditional independence test.

“ `gsq` ”: G-squared conditional independence test.

- “ `kci` ”: kernel-based conditional independence test. (As a kernel method, its complexity is cubic in the sample size, so it might be slow if the same size is not small.)

“ `kci` ”: kernel-based conditional independence test. (As a kernel method, its complexity is cubic in the sample size, so it might be slow if the same size is not small.)

- “ `mv_fisherz` ”: Missing-value Fisher’s Z conditional independence test.

“ `mv_fisherz` ”: Missing-value Fisher’s Z conditional independence test.

(For more efficient nonparametric test, you may try FastKCI and RCIT . Both implementations are still preliminary and there might be some issues.)

`alpha` : Significance level of individual partial correlation tests.
Default: 0.05.

`depth` : The depth for the fast adjacency search, or -1 if unlimited.
Default: -1.

`max_path_length` : the maximum length of any discriminating path, or -1 if unlimited. Default: -1.

`verbose` : True is verbose output should be printed or logged.
Default: False.

`background_knowledge` : class `BackgroundKnowledge`. Add prior edges according to assigned causal connections. Default: None.
For detailed usage, please kindly refer to its usage example .

cache_variables_map : This variable a map which contains the variables relate with cache. If it is not None, it should contain 'data_hash_key' 、 'ci_test_hash_key' and 'cardinalities'. Default: None.

show_progress : True iff the algorithm progress should be show in console. Default: True.

Returns

g : a GeneralGraph object, where g.graph is a PAG and the illustration of its end nodes is as follows (denotes $G = g.graph$):

- If edge.properties have the Property 'nl', then there is no latent confounder. Otherwise, there are possibly latent confounders.

If edge.properties have the Property 'nl', then there is no latent confounder. Otherwise, there are possibly latent confounders.

- If edge.properties have the Property 'dd', then it is definitely direct. Otherwise, it is possibly direct.

If edge.properties have the Property 'dd', then it is definitely direct. Otherwise, it is possibly direct.

- If edge.properties have the Property 'pl', then there are possibly latent confounders. Otherwise, there is no latent confounder.

If edge.properties have the Property 'pl', then there are possibly latent confounders. Otherwise, there is no latent confounder.

- If edge.properties have the Property 'pd', then it is possibly direct. Otherwise, it is definitely direct.

If edge.properties have the Property 'pd', then it is possibly direct. Otherwise, it is definitely direct.

Spirtes, P., Meek, C., & Richardson, T. (1995, August). Causal inference in the presence of latent variables and selection bias. In Proceedings of the Eleventh conference on Uncertainty in artificial intelligence (pp. 499-506).

CD-NOD

Algorithm Introduction

Perform Peter-Clark algorithm for causal discovery on the augmented data set that captures the unobserved changing factors (CD-NOD [1]).

Usage

```
from causallearn.search.ConstraintBased.CDNOD import cdnod #
default parameters cg = cdnod ( data , c_indx ) # or customized
parameters cg = cdnod ( data , c_indx , alpha , indep_test ,
stable , uc_rule , uc_priority , mvcdnod , correction_name ,
background_knowledge , verbose , show_progress ) # visualization
using pydot # note that the last node is the c_indx cg .
draw_pydot_graph () # or save the graph from
causallearn.utils.GraphUtils import GraphUtils pyd = GraphUtils .
to_pydot ( cg . G ) pyd . write_png ( 'simple_test.png' )
```

Visualization using pydot is recommended. If specific label names are needed, please refer to this usage example (e.g., 'cg.draw_pydot_graph(labels=["A", "B", "C"])' or 'GraphUtils.to_pydot(cg.G, labels=["A", "B", "C"])').

Parameters

data : numpy.ndarray, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

c_indx : time index or domain index that captures the unobserved changing factors.

alpha : desired significance level (float) in (0, 1). Default: 0.05.

- "fisherz": Fisher's Z conditional independence test.
- "fisherz": Fisher's Z conditional independence test.
- "chisq": Chi-squared conditional independence test.
- "chisq": Chi-squared conditional independence test.
- "gsq": G-squared conditional independence test.

“ gsq ”: G-squared conditional independence test.

- “ kci ”: kernel-based conditional independence test. (As a kernel method, its complexity is cubic in the sample size, so it might be slow if the same size is not small.)

“ kci ”: kernel-based conditional independence test. (As a kernel method, its complexity is cubic in the sample size, so it might be slow if the same size is not small.)

- “ mv_fisherz ”: Missing-value Fisher’s Z conditional independence test.

“ mv_fisherz ”: Missing-value Fisher’s Z conditional independence test.

(For more efficient nonparametric test, you may try FastKCI and RCIT . Both implementations are still preliminary and there might be some issues.)

stable : run stabilized skeleton discovery [3] if True. Default: True.

- 0: run uc_sepset.

0: run uc_sepset.

- 1: run maxP [2] . Orient an unshielded triple X-Y-Z as a collider with an additional CI test.

1: run maxP [2] . Orient an unshielded triple X-Y-Z as a collider with an additional CI test.

- 2: run definiteMaxP [2] . Orient only the definite colliders in the skeleton and keep track of all the definite non-colliders as well.

2: run definiteMaxP [2] . Orient only the definite colliders in the skeleton and keep track of all the definite non-colliders as well.

- -1: whatever is default in uc_rule.

-1: whatever is default in uc_rule.

- 0: overwrite.

0: overwrite.

- 1: orient bi-directed.

1: orient bi-directed.

- 2: prioritize existing colliders.

2: prioritize existing colliders.

- 3: prioritize stronger colliders.

3: prioritize stronger colliders.

- 4: prioritize stronger* colliders.

4: prioritize stronger* colliders.

mvp_c : use missing-value PC or not. Default (and suggested for CDNOD): False.

correction_name : Missing value correction if using missing-value PC. Default: 'MV_Crtn_Fisher_Z'

background_knowledge : class BackgroundKnowledge. Add prior edges according to assigned causal connections. Default: Nnoe. For detailed usage, please kindly refer to its usage example .

verbose : True iff verbose output should be printed. Default: False.

show_progress : True iff the algorithm progress should be show in console. Default: True.

Returns

cg : a CausalGraph object, where $cg.G.graph[j,i]=1$ and $cg.G.graph[i,j]=-1$ indicate $i \rightarrow j$; $cg.G.graph[i,j] = cg.G.graph[j,i] = -1$ indicates $i - j$; $cg.G.graph[i,j] = cg.G.graph[j,i] = 1$ indicates $i \leftrightarrow j$.

Huang, B., Zhang, K., Zhang, J., Ramsey, J. D., Sanchez-Romero, R., Glymour, C., & Schölkopf, B. (2020). Causal Discovery from Heterogeneous/Nonstationary Data. J. Mach. Learn. Res., 21(89), 1-53.

Ramsey, J. (2016). Improving accuracy and scalability of the pc algorithm by maximizing p-value. arXiv preprint arXiv:1610.00378.

Colombo, D., & Maathuis, M. H. (2014). Order-independent constraint-based causal structure learning. J. Mach. Learn. Res., 15(1), 3741-3782.

Score-based causal discovery methods

In this section, we would like to introduce Score-based causal discovery methods, including GES, DGES, and Exact search methods. For GES, we implemented it with BIC score [1] and generalized score [2] . DGES [5] extends GES to handle data with deterministic (functional) relationships among variables. For Exact search, we implemented DP [3] and A* [4] .

Contents:

- GES with the BIC score or generalized score Algorithm
Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- DGES: GES in the Presence of Deterministic Relations
Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- Exact Search Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns

Chickering, D. M. (2002). Optimal structure identification with greedy search. *Journal of machine learning research*, 3(Nov), 507-554.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 1551-1560).

Silander, T., & Myllymäki, P. (2006, July). A simple approach for finding the globally optimal Bayesian network structure. In *Proceedings of the Twenty-Second Conference on Uncertainty in Artificial Intelligence* (pp. 445-452).

Yuan, C., & Malone, B. (2013). Learning optimal Bayesian networks: A shortest path perspective. *Journal of Artificial Intelligence Research*, 48, 23-65.

GES with the BIC score or generalized score

Algorithm Introduction

Greedy Equivalence Search (GES) algorithm with BIC score [1] and generalized score [2] .

Usage

```
from causallearn.search.ScoreBased.GES import ges # default
parameters Record = ges ( X ) # or customized parameters Record =
ges ( X , score_func , maxP , parameters ) # Visualization using
pydot from causallearn.utils.GraphUtils import GraphUtils import
matplotlib.image as mpimg import matplotlib.pyplot as plt import
io pyd = GraphUtils . to_pydot ( Record [ 'G' ] ) tmp_png = pyd .
create_png ( f = "png" ) fp = io . BytesIO ( tmp_png ) img =
mpimg . imread ( fp , format = 'png' ) plt . axis ( 'off' ) plt .
imshow ( img ) plt . show () # or save the graph pyd . write_png
( 'simple_test.png' )
```

Visualization using pydot is recommended (usage example). If specific label names are needed, please refer to this usage example (e.g., `GraphUtils.to_pydot(Record['G'], labels=["A", "B", "C"])`).

Parameters

X : `numpy.ndarray`, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

- " local_score_BIC ": BIC score [3] .

" local_score_BIC ": BIC score [3] .

- " local_score_BDeu ": BDeu score [4] .

“ local_score_BDeu ”: BDeu score [4] .

- “ local_score_CV_general ”: Generalized score with cross validation for data with single-dimensional variables [2] .

“ local_score_CV_general ”: Generalized score with cross validation for data with single-dimensional variables [2] .

- “ local_score_marginal_general ”: Generalized score with marginal likelihood for data with single-dimensional variables [2] .

“ local_score_marginal_general ”: Generalized score with marginal likelihood for data with single-dimensional variables [2] .

- “ local_score_CV_multi ”: Generalized score with cross validation for data with multi-dimensional variables [2] .

“ local_score_CV_multi ”: Generalized score with cross validation for data with multi-dimensional variables [2] .

- “ local_score_marginal_multi ”: Generalized score with marginal likelihood for data with multi-dimensional variables [2] .

“ local_score_marginal_multi ”: Generalized score with marginal likelihood for data with multi-dimensional variables [2] .

maxP : Allowed maximum number of parents when searching the graph. Default: None.

- parameters[‘kfold’]: k-fold cross validation.

parameters[‘kfold’]: k-fold cross validation.

- parameters[‘lambda’]: regularization parameter.

parameters[‘lambda’]: regularization parameter.

- parameters[‘dlabel’]: for variables with multi-dimensions, indicate which dimensions belong to the i-th variable.

parameters[‘dlabel’]: for variables with multi-dimensions, indicate which dimensions belong to the i-th variable.

Returns

- Record[‘G’] : learned causal graph, where Record[‘G’].graph[j,i]=1 and Record[‘G’].graph[i,j]=-1 indicate $i \rightarrow j$; Record[‘G’].graph[i,j] = Record[‘G’].graph[j,i] = -1 indicates $i - j$.

Record['G'] : learned causal graph, where
Record['G'].graph[j,i]=1 and Record['G'].graph[i,j]=-1 indicate $i \rightarrow j$; Record['G'].graph[i,j] = Record['G'].graph[j,i] = -1 indicates $i \dashv j$.

- Record['update1'] : each update (Insert operator) in the forward step.

Record['update1'] : each update (Insert operator) in the forward step.

- Record['update2'] : each update (Delete operator) in the backward step.

Record['update2'] : each update (Delete operator) in the backward step.

- Record['G_step1'] : learned graph at each step in the forward step.

Record['G_step1'] : learned graph at each step in the forward step.

- Record['G_step2'] : learned graph at each step in the backward step.

Record['G_step2'] : learned graph at each step in the backward step.

- Record['score'] : the score of the learned graph.

Record['score'] : the score of the learned graph.

Chickering, D. M. (2002). Optimal structure identification with greedy search. *Journal of machine learning research*, 3(Nov), 507-554.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 1551-1560).

Schwarz, G. (1978). Estimating the dimension of a model. *The annals of statistics*, 461-464.

Buntine, W. (1991). Theory refinement on Bayesian networks. In *Uncertainty proceedings 1991* (pp. 52-60). Morgan Kaufmann.

DGES: GES in the Presence of Deterministic Relations

Algorithm Introduction

Deterministic Greedy Equivalence Search (DGES) [1] extends GES to handle data containing deterministic (functional) relationships among variables. Standard GES can fail when some variables are deterministic functions of others, because the standard BIC score becomes degenerate (the residual variance is zero). DGES addresses this by detecting Minimal Deterministic Clusters (MinDCs) from the data and incorporating them into the search procedure.

DGES implements Algorithm 1 from the paper, which consists of three phases:

- Phase 1 — Detect MinDCs : Identify sets of variables where one variable is a deterministic function of the others, using eigenvalue analysis of the covariance matrix.

Phase 1 — Detect MinDCs : Identify sets of variables where one variable is a deterministic function of the others, using eigenvalue analysis of the covariance matrix.

- Phase 2 — DC-aware GES : Run a modified GES that forces edges between variables within the same MinDC during the forward phase and protects these edges from deletion during the backward phase. This uses a deterministic-aware BIC score that handles zero residual variance gracefully.

Phase 2 — DC-aware GES : Run a modified GES that forces edges between variables within the same MinDC during the forward phase and protects these edges from deletion during the backward phase. This uses a deterministic-aware BIC score that handles zero residual variance gracefully.

- Phase 3 — Exact Search (optional) : Perform exact search on deterministic cluster nodes and their neighbors to refine the graph locally. This phase is skipped by default due to its computational cost.

Phase 3 — Exact Search (optional) : Perform exact search on deterministic cluster nodes and their neighbors to refine the graph locally. This phase is skipped by default due to its computational cost.

When no deterministic relations are present in the data, DGES behaves identically to standard GES.

Usage

```
from causallearn.search.ScoreBased.DGES import dges # Basic usage
with default deterministic BIC score Record = dges ( X ) # With
standard BIC score (for non-deterministic data) Record = dges
( X , score_func = 'local_score_BIC' ) # With Phase 3 exact search
enabled Record = dges ( X , skip_exact_search = False ) # With
custom parameters Record = dges ( X , score_func =
'local_score_BIC' , parameters = { 'lambda_value' : 2.0 },
node_names = [ 'X1' , 'X2' , 'X3' ]) # Visualization using pydot
from causallearn.utils.GraphUtils import GraphUtils import
matplotlib.image as mpimg import matplotlib.pyplot as plt import
io pyd = GraphUtils . to_pydot ( Record [ 'G' ]) tmp_png = pyd .
create_png ( f = "png" ) fp = io . BytesIO ( tmp_png ) img =
mpimg . imread ( fp , format = 'png' ) plt . axis ( 'off' ) plt .
imshow ( img ) plt . show ()
```

Parameters

X : numpy.ndarray, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

- 'local_score_BIC_from_cov_deterministic' : Deterministic-aware BIC score that handles zero residual variance (recommended for data with deterministic relations).

'local_score_BIC_from_cov_deterministic' : Deterministic-aware BIC score that handles zero residual variance (recommended for data with deterministic relations).

- 'local_score_BIC' : Standard BIC score (use when no deterministic relations are present).

'local_score_BIC' : Standard BIC score (use when no deterministic relations are present).

- 'local_score_BDeu' : BDeu score.

'local_score_BDeu' : BDeu score.

maxP : Allowed maximum number of parents when searching the graph. Default: None (uses sample size N).

- parameters['lambda_value']: Penalty hyperparameter for BIC score. Default: 0.5.

parameters['lambda_value']: Penalty hyperparameter for BIC score. Default: 0.5.

`node_names` : list of str or None. Custom names for graph nodes.
Default: None (uses X1, X2, ...).

`det_threshold` : float. Threshold on eigenvalue ratio for detecting deterministic relations. Smaller values require stricter determinism. Default: 1e-6 .

`det_epsilon` : float. Small constant added to residual variance in the deterministic BIC score to avoid log(0). Default: 1e-8 .

- `'dp'` : Dynamic programming.

`'dp'` : Dynamic programming.

- `'astar'` : A* search.

`'astar'` : A* search.

`skip_exact_search` : bool. If True, skip Phase 3 (exact search) and return the result after Phase 2. Default: True .

Returns

- `Record['G']` : GeneralGraph. The learned causal graph (CPDAG), where `Record['G'].graph[j,i]=1` and `Record['G'].graph[i,j]=-1` indicate $i \rightarrow j$; `Record['G'].graph[i,j] = Record['G'].graph[j,i] = -1` indicates $i - j$.

`Record['G']` : GeneralGraph. The learned causal graph (CPDAG), where `Record['G'].graph[j,i]=1` and `Record['G'].graph[i,j]=-1` indicate $i \rightarrow j$; `Record['G'].graph[i,j] = Record['G'].graph[j,i] = -1` indicates $i - j$.

- `Record['score']` : float. The score of the learned graph.

`Record['score']` : float. The score of the learned graph.

- `Record['update1']` : each update (Insert operator) in the forward step.

`Record['update1']` : each update (Insert operator) in the forward step.

- `Record['update2']` : each update (Delete operator) in the backward step.

`Record['update2']` : each update (Delete operator) in the backward step.

- `Record['G_step1']` : learned graph at each step in the forward step.

Record['G_step1'] : learned graph at each step in the forward step.

- Record['G_step2'] : learned graph at each step in the backward step.

Record['G_step2'] : learned graph at each step in the backward step.

- Record['mindcs'] : list of lists. Detected Minimal Deterministic Clusters. Each inner list contains the variable indices forming a MinDC.

Record['mindcs'] : list of lists. Detected Minimal Deterministic Clusters. Each inner list contains the variable indices forming a MinDC.

- Record['mindc_sets'] : list of sets. Same as mindcs but as sets.

Record['mindc_sets'] : list of sets. Same as mindcs but as sets.

- Record['det_clusters'] : list of sets. Deterministic cluster groups (unions of overlapping MinDCs).

Record['det_clusters'] : list of sets. Deterministic cluster groups (unions of overlapping MinDCs).

- Record['exact_search_nodes'] : list. Nodes included in Phase 3 exact search (empty if skip_exact_search=True).

Record['exact_search_nodes'] : list. Nodes included in Phase 3 exact search (empty if skip_exact_search=True).

Li, L., Dai, H., Al Ghothani, H., Huang, B., Zhang, J., Harel, S., ... & Zhang, K. (2024). On causal discovery in the presence of deterministic relations. *Advances in Neural Information Processing Systems*, 37, 130920-130952.

Exact Search

Algorithm Introduction

Search for the optimal graph using Dynamic Programming (DP [1]) or A* search [2] .

Usage

```
from causallearn.search.ScoreBased.ExactSearch import
bic_exact_search dag_est , search_stats = bic_exact_search ( X ,
super_graph , search_method , use_path_extension ,
use_k_cycle_heuristic , k , verbose , include_graph ,
max_parents )
```

Parameters

X : numpy.ndarray, shape=(n, d). The data to fit the structure too, where each row is a sample and each column corresponds to the associated variable.

super_graph : numpy.ndarray, shape=(d, d). Super-structure to restrict search space (binary matrix). If None, no super-structure is used. Default is None.

search_method : str. Method of exact search (['astar', 'dp']). Default is astar.

use_path_extension : bool. Whether to use optimal path extension for order graph. Note that this trick will not affect the correctness of search procedure. Default is True.

use_k_cycle_heuristic : bool. Whether to use k-cycle conflict heuristic for astar. Default is False.

k : int. Parameter used by k-cycle conflict heuristic for astar. Default is 3.

verbose : bool. Whether to log messages related to search procedure.

max_parents : int. The maximum number of parents a node can have. If used, this means using the k-learn procedure. Can drastically speed up algorithms. If None, no max on parents. Default is None.

Returns

dag_est : numpy.ndarray, shape=(d, d). Estimated DAG.

search_stats : dict. Some statistics related to the search procedure.

Silander, T., & Myllymäki, P. (2006, July). A simple approach for finding the globally optimal Bayesian network structure. In Proceedings of the Twenty-Second Conference on Uncertainty in Artificial Intelligence (pp. 445-452).

Yuan, C., & Malone, B. (2013). Learning optimal Bayesian networks: A shortest path perspective. Journal of Artificial Intelligence Research, 48, 23-65.

Causal discovery methods based on constrained functional causal models

In this section, we would like to introduce causal discovery methods based on constrained functional causal models. Now we have LiNGAM-based methods (ICA-based LiNGAM [1], DirectLiNGAM [2], VAR-LiNGAM [3], RCD [4], and CAM-UV [5]), post-nonlinear (PNL [6]) causal models, and additive noise models (ANM [7]).

Contents:

- LiNGAM-based Methods ICA-based LiNGAM Parameters Returns DirectLiNGAM Parameters Returns VAR-LiNGAM Parameters Returns RCD Parameters Returns CAM-UV Parameters Returns
- ICA-based LiNGAM Parameters Returns
- Parameters
- Returns
- DirectLiNGAM Parameters Returns
- Parameters
- Returns
- VAR-LiNGAM Parameters Returns
- Parameters
- Returns
- RCD Parameters Returns
- Parameters
- Returns
- CAM-UV Parameters Returns
- Parameters
- Returns
- Post-nonlinear causal models Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns

- Additive noise models Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns

Shimizu, S., Hoyer, P. O., Hyvärinen, A., Kerminen, A., & Jordan, M. (2006). A linear non-Gaussian acyclic model for causal discovery. *Journal of Machine Learning Research*, 7(10).

Shimizu, S., Inazumi, T., Sogawa, Y., Hyvärinen, A., Kawahara, Y., Washio, T., ... & Bollen, K. (2011). DirectLiNGAM: A direct method for learning a linear non-Gaussian structural equation model. *The Journal of Machine Learning Research*, 12, 1225-1248.

Hyvärinen, A., Zhang, K., Shimizu, S., & Hoyer, P. O. (2010). Estimation of a structural vector autoregression model using non-gaussianity. *Journal of Machine Learning Research*, 11(5).

Maeda, T. N., & Shimizu, S. (2020, June). RCD: Repetitive causal discovery of linear non-Gaussian acyclic models with latent confounders. In *International Conference on Artificial Intelligence and Statistics* (pp. 735-745). PMLR.

Maeda, T. N., & Shimizu, S. (2021). Causal Additive Models with Unobserved Variables. *UAI*.

Zhang, K., & Hyvärinen, A. (2009, June). On the Identifiability of the Post-Nonlinear Causal Model. In *25th Conference on Uncertainty in Artificial Intelligence (UAI 2009)* (pp. 647-655). AUAI Press.

Hoyer, P. O., Janzing, D., Mooij, J. M., Peters, J., & Schölkopf, B. (2008, December). Nonlinear causal discovery with additive noise models. In *NIPS* (Vol. 21, pp. 689-696).

LiNGAM-based Methods

Estimation of Linear, Non-Gaussian Acyclic Model from observed data. It assumes non-Gaussianity of the noise terms in the causal model.

causal-learn has the official implementations for a set of LiNGAM-based methods (e.g., ICA-based LiNGAM [1], DirectLiNGAM [2], VAR-LiNGAM [3], RCD [4], and CAM-UV [5]). And we are actively updating the list.

Shimizu, S., Hoyer, P. O., Hyvärinen, A., Kerminen, A., & Jordan, M. (2006). A linear non-Gaussian acyclic model for causal discovery. *Journal of Machine Learning Research*, 7(10).

Shimizu, S., Inazumi, T., Sogawa, Y., Hyvärinen, A., Kawahara, Y., Washio, T., ... & Bollen, K. (2011). DirectLiNGAM: A direct method for learning a linear non-Gaussian structural equation model. *The Journal of Machine Learning Research*, 12, 1225-1248.

Hyvärinen, A., Zhang, K., Shimizu, S., & Hoyer, P. O. (2010). Estimation of a structural vector autoregression model using non-gaussianity. *Journal of Machine Learning Research*, 11(5).

Maeda, T. N., & Shimizu, S. (2020, June). RCD: Repetitive causal discovery of linear non-Gaussian acyclic models with latent confounders. In *International Conference on Artificial Intelligence and Statistics* (pp. 735-745). PMLR.

Maeda, T. N., & Shimizu, S. (2021). Causal Additive Models with Unobserved Variables. *UAI*.

ICA-based LiNGAM

```
from causallearn.search.FCMBased import lingam model = lingam .  
ICALiNGAM ( random_state , max_iter ) model . fit ( X ) print  
( model . causal_order_ ) print ( model . adjacency_matrix_ )
```

Parameters

`random_state` : int, optional (default=None). The seed used by the random number generator.

`max_iter` : int, optional (default=1000). The maximum number of iterations of FastICA.

`X` : array-like, shape (n_samples, n_features). Training data, where n_samples is the number of samples and n_features is the number of features.

Returns

`model.causal_order_` : array-like, shape (n_features). The causal order of fitted model, where n_features is the number of features.

`model.adjacency_matrix_` : array-like, shape (n_features, n_features). The adjacency matrix B of fitted model, where n_features is the number of features.

DirectLiNGAM

```
from causallearn.search.FCMBased import lingam
model = lingam.DirectLiNGAM ( random_state , prior_knowledge ,
                               apply_prior_knowledge_softly , measure )
model . fit ( X )
print ( model . causal_order_ )
print ( model . adjacency_matrix_ )
```

Parameters

`random_state` : int, optional (default=None). The seed used by the random number generator.

`prior_knowledge` : array-like, shape (n_features, n_features), optional (default=None). Prior knowledge used for causal discovery, where n_features is the number of features. The elements of prior knowledge matrix are defined as follows:

- 0: (x_i) does not have a directed path to (x_j)

0: (x_i) does not have a directed path to (x_j)

- 1: (x_i) has a directed path to (x_j)

1: (x_i) has a directed path to (x_j)

- -1: No prior knowledge is available to know if either of the two cases above (0 or 1) is true.

-1: No prior knowledge is available to know if either of the two cases above (0 or 1) is true.

`apply_prior_knowledge_softly` : boolean, optional (default=False). If True, apply prior knowledge softly.

`measure` : {'pwling', 'kernel'}, optional (default='pwling'). Measure to evaluate independence: 'pwling' or 'kernel'.

`X` : array-like, shape (n_samples, n_features). Training data, where n_samples is the number of samples and n_features is the number of features.

Returns

`model.causal_order_` : array-like, shape (n_features). The causal order of fitted model, where n_features is the number of features.

`model.adjacency_matrix_` : array-like, shape (n_features, n_features). The adjacency matrix B of fitted model, where n_features is the number of features.

VAR-LiNGAM

```
from causallearn.search.FCMBased import lingam model = lingam .  
VARLiNGAM ( lags , criterion , prune , ar_coefs , lingam_model ,  
random_state ) model . fit ( X ) print ( model . causal_order_  
print ( model . adjacency_matrices_ [ 0 ]) print ( model .  
adjacency_matrices_ [ 1 ]) print ( model . residuals_ )
```

Parameters

lags : int, optional (default=1). Number of lags.

criterion : {'aic', 'fpe', 'hqic', 'bic', None}, optional (default='bic'). Criterion to decide the best lags within 'lags'. Searching the best lags is disabled if 'criterion' is None.

prune : boolean, optional (default=False). Whether to prune the adjacency matrix or not.

ar_coefs : array-like, optional (default=None). Coefficients of AR model. Estimating AR model is skipped if specified 'ar_coefs'. Shape must be ('lags', n_features, n_features).

lingam_model : lingam object inherits 'lingam._BaseLiNGAM', optional (default=None). LiNGAM model for causal discovery. If None, DirectLiNGAM algorithm is selected.

random_state : int, optional (default=None). 'random_state' is the seed used by the random number generator.

X : array-like, shape (n_samples, n_features). Training data, where n_samples is the number of samples and n_features is the number of features.

Returns

model.causal_order_ : array-like, shape (n_features). The causal order of fitted model, where n_features is the number of features.

model.adjacency_matrices_ : array-like, shape (lags, n_features, n_features). The adjacency matrix of fitted model, where n_features is the number of features.

model.residuals_ : array-like, shape (n_samples). Residuals of regression, where n_samples is the number of samples.

RCD

```
from causallearn.search.FCMBased import lingam model = lingam .  
RCD ( max_explanatory_num , cor_alpha , ind_alpha ,
```

```
shapiro_alpha , MLHSICR , bw_method ) model . fit ( X ) print
( model . adjacency_matrix_ ) print ( model . ancestors_list_ )
```

Parameters

`max_explanatory_num` : int, optional (default=2). Maximum number of explanatory variables.

`cor_alpha` : float, optional (default=0.01). Alpha level for pearson correlation.

`ind_alpha` : float, optional (default=0.01). Alpha level for HSIC.

`shapiro_alpha` : float, optional (default=0.01). Alpha level for Shapiro-Wilk test.

`MLHSICR` : bool, optional (default=False). If True, use MLHSICR for multiple regression, if False, use OLS for multiple regression.

- ‘mdbs’: Median distance between samples.

‘mdbs’: Median distance between samples.

- ‘scott’: Scott’s Rule of Thumb.

‘scott’: Scott’s Rule of Thumb.

- ‘silverman’: Silverman’s Rule of Thumb.

‘silverman’: Silverman’s Rule of Thumb.

`X` : array-like, shape (n_samples, n_features). Training data, where n_samples is the number of samples and n_features is the number of features.

Returns

`model.adjacency_matrix_` : array-like, shape (n_features, n_features). The adjacency matrix B of fitted model, where n_features is the number of features.

`model.ancestors_list_` : array-like, shape (n_features). The list of causal ancestors sets, where n_features is the number of features.

CAM-UV

```
from causallearn.search.FCMBased.lingam import CAMUV
P , U = CAMUV . execute ( data , alpha , num_explanatory_vals )
for i , result in enumerate ( P ):
    if not len ( result ) == 0 :
        print
```

```
( "child: " + str ( i ) + ", parents: " + str ( result )) for  
result in U : print ( result )
```

Parameters

data : array-like, shape (n_samples, n_features). Training data, where n_samples is the number of samples and n_features is the number of features.

alpha : the alpha level for independence testing.

num_explanatory_vals : the maximum number of variables to infer causal relationships. This is equivalent to d in the paper.

Returns

P : P[i] contains the indices of the parents of Xi.

U : The indices of variable pairs having UCPs or UBPs.

Post-nonlinear causal models



Algorithm Introduction

Causal discovery based on the post-nonlinear (PNL [1]) causal models. If you would like to apply the method to more than two variables, we suggest you first apply the PC algorithm and then use pair-wise analysis in this implementation to find the causal directions that cannot be determined by PC.

Usage

```
from causallearn.search.FCMBased.PNL.PNL import PNL pnl = PNL ()  
p_value_foward , p_value_backward = pnl . cause_or_effect  
( data_x , data_y )
```

Parameters

data_x : input data (n, 1), n is the sample size.

data_y : output data (n, 1), n is the sample size.

Returns

pval_forward : p value in the x->y direction.

pval_backward : p value in the y->x direction.

Zhang, K., & Hyvärinen, A. (2009, June). On the Identifiability of the Post-Nonlinear Causal Model. In 25th Conference on Uncertainty in Artificial Intelligence (UAI 2009) (pp. 647-655). AUAI Press.

Additive noise models

Algorithm Introduction

Causal discovery based on the additive noise models (ANM [1]). If you would like to apply the method to more than two variables, we suggest you first apply the PC algorithm and then use pair-wise analysis in this implementation to find the causal directions that cannot be determined by PC.

Usage

```
from causalllearn.search.FCMBased.ANM.ANM import ANM anm = ANM ()  
p_value_foward , p_value_backward = anm . cause_or_effect  
( data_x , data_y )
```

Parameters

data_x : input data (n, 1).

data_y : output data (n, 1).

Returns

pval_forward : p value in the x->y direction.

pval_backward : p value in the y->x direction.

Hoyer, P. O., Janzing, D., Mooij, J. M., Peters, J., & Schölkopf, B. (2008, December). Nonlinear causal discovery with additive noise models. In NIPS (Vol. 21, pp. 689-696).

Hidden causal representation learning

In this section, we would like to introduce methods in hidden causal representation learning, such as generalized independent noise (GIN [1]) condition-based method.

Contents:

- Generalized Independence Noise (GIN) condition-based method Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns

Xie, F., Cai, R., Huang, B., Glymour, C., Hao, Z., & Zhang, K. (2020, January). Generalized Independent Noise Condition for Estimating Latent Variable Causal Graphs. In NeurIPS.

Generalized Independence Noise (GIN) condition-based method

Algorithm Introduction

Learning the structure of Linear, Non-Gaussian LATent variable Model (LiNLAM) based the GIN [1] condition.

Usage

```
from causallearn.search.HiddenCausal.GIN.GIN import GIN G , K =
GIN ( data ) # Visualization using pydot from
causallearn.utils.GraphUtils import GraphUtils import
matplotlib.image as mpimg import matplotlib.pyplot as plt import
io pyd = GraphUtils . to_pydot ( G ) tmp_png = pyd . create_png
( f = "png" ) fp = io . BytesIO ( tmp_png ) img = mpimg . imread
( fp , format = 'png' ) plt . axis ( 'off' ) plt . imshow ( img )
plt . show ()
```

Visualization using pydot is recommended (usage example). If specific label names are needed, please refer to this usage example (e.g., `GraphUtils.to_pydot(G, labels=["A", "B", "C"])`).

Parameters

`data` : `numpy.ndarray`, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

Returns

`G` : `GeneralGraph`. Causal graph.

`K` : list. Causal Order.

Xie, F., Cai, R., Huang, B., Glymour, C., Hao, Z., & Zhang, K. (2020, January). Generalized Independent Noise Condition for Estimating Latent Variable Causal Graphs. In NeurIPS.

Permutation-based causal discovery methods

In this section, we would like to introduce methods based on permutations, such as GRaSP [1] .

Contents:

- GRaSP Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns
- BOSS Algorithm Introduction Usage Parameters Returns
- Algorithm Introduction
- Usage
- Parameters
- Returns

Lam, W. Y., Andrews, B., & Ramsey, J. (2022, February). Greedy Relaxations of the Sparsest Permutation Algorithm. In The 38th Conference on Uncertainty in Artificial Intelligence.

GRaSP

Algorithm Introduction

Greedy relaxations of the sparsest permutation (GRaSP) algorithm [1] .

Usage

```
from causallearn.search.PermutationBased.GRaSP import grasp #
default parameters G = grasp ( X ) # or customized parameters G =
grasp ( X , score_func , depth , parameters ) # Visualization
using pydot from causallearn.utils.GraphUtils import GraphUtils
import matplotlib.image as mpimg import matplotlib.pyplot as plt
import io pyd = GraphUtils . to_pydot ( G ) tmp_png = pyd .
create_png ( f = "png" ) fp = io . BytesIO ( tmp_png ) img =
mpimg . imread ( fp , format = 'png' ) plt . axis ( 'off' ) plt .
imshow ( img ) plt . show ()
```

Visualization using pydot is recommended (usage example). If specific label names are needed, please refer to this usage example (e.g., `GraphUtils.to_pydot(G, labels=["A", "B", "C"])`).

Parameters

`X` : `numpy.ndarray`, shape (`n_samples`, `n_features`). Data, where `n_samples` is the number of samples and `n_features` is the number of features.

- “ `local_score_BIC` ”: BIC score [3] .

“ `local_score_BIC` ”: BIC score [3] .

- “ `local_score_BDeu` ”: BDeu score [4] .

“ `local_score_BDeu` ”: BDeu score [4] .

- “ `local_score_CV_general` ”: Generalized score with cross validation for data with single-dimensional variables [2] .

“ `local_score_CV_general` ”: Generalized score with cross validation for data with single-dimensional variables [2] .

- “ `local_score_marginal_general` ”: Generalized score with marginal likelihood for data with single-dimensional variables [2] .

“ local_score_marginal_general ”: Generalized score with marginal likelihood for data with single-dimensional variables [2] .

- “ local_score_CV_multi ”: Generalized score with cross validation for data with multi-dimensional variables [2] .

“ local_score_CV_multi ”: Generalized score with cross validation for data with multi-dimensional variables [2] .

- “ local_score_marginal_multi ”: Generalized score with marginal likelihood for data with multi-dimensional variables [2] .

“ local_score_marginal_multi ”: Generalized score with marginal likelihood for data with multi-dimensional variables [2] .

- parameters[‘kfold’]: k-fold cross validation.

parameters[‘kfold’]: k-fold cross validation.

- parameters[‘lambda’]: regularization parameter.

parameters[‘lambda’]: regularization parameter.

- parameters[‘dlabel’]: for variables with multi-dimensions, indicate which dimensions belong to the i-th variable.

parameters[‘dlabel’]: for variables with multi-dimensions, indicate which dimensions belong to the i-th variable.

Returns

- G : learned general graph, where $G.\text{graph}[j,i]=1$ and $G.\text{graph}[i,j]=-1$ indicate $i \rightarrow j$; $G.\text{graph}[i,j] = G.\text{graph}[j,i] = -1$ indicates $i \dashv j$.

G : learned general graph, where $G.\text{graph}[j,i]=1$ and $G.\text{graph}[i,j]=-1$ indicate $i \rightarrow j$; $G.\text{graph}[i,j] = G.\text{graph}[j,i] = -1$ indicates $i \dashv j$.

Lam, W. Y., Andrews, B., & Ramsey, J. (2022, February). Greedy Relaxations of the Sparsest Permutation Algorithm. In The 38th Conference on Uncertainty in Artificial Intelligence.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 1551-1560).

Schwarz, G. (1978). Estimating the dimension of a model. The annals of statistics, 461-464.

BOSS

Algorithm Introduction

Best order score search (BOSS) algorithm [1] .

Usage

```
from causallearn.search.PermutationBased.BOSS import boss #
default parameters G = boss ( X ) # or customized parameters G =
boss ( X , score_func , parameters ) # Visualization using pydot
from causallearn.utils.GraphUtils import GraphUtils import
matplotlib.image as mpimg import matplotlib.pyplot as plt import
io pyd = GraphUtils . to_pydot ( G ) tmp_png = pyd . create_png
( f = "png" ) fp = io . BytesIO ( tmp_png ) img = mpimg . imread
( fp , format = 'png' ) plt . axis ( 'off' ) plt . imshow ( img )
plt . show ()
```

Visualization using pydot is recommended (usage example). If specific label names are needed, please refer to this usage example (e.g., GraphUtils.to_pydot(G, labels=["A", "B", "C"])).

Parameters

X : numpy.ndarray, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

- “ local_score_BIC ”: BIC score [3] .
- “ local_score_BIC ”: BIC score [3] .
- “ local_score_BDeu ”: BDeu score [4] .
- “ local_score_BDeu ”: BDeu score [4] .
- “ local_score_CV_general ”: Generalized score with cross validation for data with single-dimensional variables [2] .

“ local_score_CV_general ”: Generalized score with cross validation for data with single-dimensional variables [2] .

- “ local_score_marginal_general ”: Generalized score with marginal likelihood for data with single-dimensional variables [2] .

“ local_score_marginal_general ”: Generalized score with marginal likelihood for data with single-dimensional variables [2] .

- “ local_score_CV_multi ”: Generalized score with cross validation for data with multi-dimensional variables [2] .

“ local_score_CV_multi ”: Generalized score with cross validation for data with multi-dimensional variables [2] .

- “ local_score_marginal_multi ”: Generalized score with marginal likelihood for data with multi-dimensional variables [2] .

“ local_score_marginal_multi ”: Generalized score with marginal likelihood for data with multi-dimensional variables [2] .

- parameters[‘kfold’]: k-fold cross validation.

parameters[‘kfold’]: k-fold cross validation.

- parameters[‘lambda’]: regularization parameter.

parameters[‘lambda’]: regularization parameter.

- parameters[‘dlabel’]: for variables with multi-dimensions, indicate which dimensions belong to the i-th variable.

parameters[‘dlabel’]: for variables with multi-dimensions, indicate which dimensions belong to the i-th variable.

Returns

- G : learned general graph, where $G.\text{graph}[j,i]=1$ and $G.\text{graph}[i,j]=-1$ indicate $i \rightarrow j$; $G.\text{graph}[i,j] = G.\text{graph}[j,i] = -1$ indicates $i - j$.

G : learned general graph, where $G.\text{graph}[j,i]=1$ and $G.\text{graph}[i,j]=-1$ indicate $i \rightarrow j$; $G.\text{graph}[i,j] = G.\text{graph}[j,i] = -1$ indicates $i - j$.

Andrews, B., Ramsey, J., Sanchez Romero, R., Camchong, J., & Kummerfeld, E. (2023). Fast scalable and accurate discovery of dags using the best order score search and grow shrink trees. *Advances in Neural Information Processing Systems*, 36, 63945-63956.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 1551-1560).

Schwarz, G. (1978). Estimating the dimension of a model. *The annals of statistics*, 461-464.

Buntine, W. (1991). Theory refinement on Bayesian networks. In *Uncertainty proceedings 1991* (pp. 52-60). Morgan Kaufmann.

Granger causality

In this section, we would like to introduce methods in Granger Causality. Now we have linear Granger Causality.

Contents:

- Linear granger causality Algorithm Introduction Usage Parameters Returns
 - Algorithm Introduction
 - Usage
 - Parameters
 - Returns
-

Linear granger causality

Algorithm Introduction

Implementation of granger causality [1] , including 1) regression+hypothesis test and 2) lasso regression [2] .

Usage

```
from causallearn.search.Granger.Granger import Granger
G = Granger()
p_value_matrix = G . granger_test_2d ( data )
coeff = G . granger_lasso ( data )
```

Parameters

`data` : `numpy.ndarray`, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features. Note that for `granger_test_2d()`, the shape of input data is (n_samples, 2).

Returns

`p_value_matrix` : p values for $x_1 \rightarrow x_2$ and $x_2 \rightarrow x_1$ (for 'granger_test_2d', which is the granger causality test for two-dimensional time series).

`coeff` : coefficient matrix (for 'granger_lasso', which is the granger causality test for multi-dimensional time series).

Granger, C. W. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica: journal of the Econometric Society*, 424-438.

Shojaie, Ali, and George Michailidis. "Discovering graphical Granger causality using the truncating lasso penalty." *Bioinformatics* 26.18 (2010): i517-i523.

(Conditional) independence tests

In this section, we would like to introduce (conditional) independence tests in causal-learn. Currently we have Fisher-z test [1], Missing-value Fisher-z test, Chi-Square test, Kernel-based conditional independence (KCI) test and independence test [2], and G-Square test [3].

(For more efficient nonparametric test, you may try FastKCI and RCIT . Both implementations are still preliminary and there might be some issues.)

Contents:

- Fisher-z test
- Missing-value Fisher-z test
- Chi-Square test
- Kernel-based conditional independence (KCI) test and independence test
- G-Square test

Fisher, R. A. (1921). On the 'probable error' of a coefficient of correlation deduced from a small sample. *Metron*, 1, 1-32.

Zhang, K., Peters, J., Janzing, D., & Schölkopf, B. (2011, July). Kernel-based Conditional Independence Test and Application in Causal Discovery. In 27th Conference on Uncertainty in Artificial Intelligence (UAI 2011) (pp. 804-813). AUAI Press.

Tsamardinos, I., Brown, L. E., & Aliferis, C. F. (2006). The max-min hill-climbing Bayesian network structure learning algorithm. *Machine learning*, 65(1), 31-78.

Fisher-z test

Perform an independence test using Fisher-z's test [1] . This test is optimal for linear-Gaussian data.

Usage

```
from causallearn.utils.cit import CIT
fisherz_obj = CIT ( data ,
"fisherz" ) # construct a CIT instance with data and method name
pValue = fisherz_obj ( X , Y , S )
```

Please be kindly informed that we have refactored the independence tests from functions to classes since the release v0.1.2.8 . Speed gain and a more flexible parameters specification are enabled.

For users, you may need to adjust your codes accordingly. Specifically,

- If you are running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

If you are running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

```
from causallearn.search.ConstraintBased.PC import pc from
causallearn.utils.cit import fisherz cg = pc ( data , 0.05 ,
fisherz )
```

- If you are explicitly calculating the p-value of a test: then you need to declare the `fisherz_obj` and then call it as above, instead of using `fisherz(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.fisherz` is a string “fisherz” , instead of a function.

If you are explicitly calculating the p-value of a test: then you need to declare the `fisherz_obj` and then call it as above, instead of using `fisherz(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.fisherz` is a string “fisherz” , instead of a function.

Please see CIT.py for more details on the implementation of the (conditional) independent tests.

Parameters

`data` : `numpy.ndarray`, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

`method` : string, “fisherz”.

`kwargs` : e.g., `cache_path` . See Advanced Usages .

Returns

`p` : the p-value of the test.

Fisher, R. A. (1921). On the ‘probable error’ of a coefficient of correlation deduced from a small sample. *Metron*, 1, 1-32.

Missing-value Fisher-z test

Perform a testwise-deletion Fisher-z independence test to data sets with missing values. With testwise-deletion, the test makes use of all data points that do not have missing values for the variables involved in the test.

Usage

```
from causallearn.utils.cit import CIT mv_fisherz_obj = CIT  
( data_with_missingness , "mv_fisherz" ) # construct a CIT  
instance with data and method name pValue = mv_fisherz_obj ( X ,  
Y , S )
```

Please be kindly informed that we have refactored the independence tests from functions to classes since the release v0.1.2.8 . Speed gain and a more flexible parameters specification are enabled.

For users, you may need to adjust your codes accordingly. Specifically, if you are

- running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

```
from causallearn.search.ConstraintBased.PC import pc from  
causallearn.utils.cit import mv_fisherz cg = pc  
( data_with_missingness , 0.05 , mv_fisherz )
```

- explicitly calculating the p-value of a test: then you need to declare the `mv_fisherz_obj` and then call it as above, instead of using `mv_fisherz(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.mv_fisherz` is a string "mv_fisherz" , instead of a function.

explicitly calculating the p-value of a test: then you need to declare the `mv_fisherz_obj` and then call it as above, instead of using `mv_fisherz(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.mv_fisherz` is a string "mv_fisherz" , instead of a function.

Please see CIT.py for more details on the implementation of the (conditional) independent tests.

Parameters

`data` : numpy.ndarray, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

`method` : string, "mv_fisherz".

kwargs : e.g., cache_path . See Advanced Usages .

Returns

p : the p-value of the test.

Chi-Square test

Perform an independence test on discrete variables using Chi-Square test.

Usage

```
from causallearn.utils.cit import CIT
chisq_obj = CIT ( data ,
"chisq" ) # construct a CIT instance with data and method name
pValue = chisq_obj ( X , Y , S )
```

Please be kindly informed that we have refactored the independence tests from functions to classes since the release v0.1.2.8 . Speed gain and a more flexible parameters specification are enabled.

For users, you may need to adjust your codes accordingly. Specifically, if you are

- running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

```
from causallearn.search.ConstraintBased.PC import pc
from causallearn.utils.cit import chisq
cg = pc ( data , 0.05 , chisq )
```

- explicitly calculating the p-value of a test: then you need to declare the chisq_obj and then call it as above, instead of using chisq(data, X, Y, condition_set) as before. Note that now causallearn.utils.cit.chisq is a string "chisq" , instead of a function.

explicitly calculating the p-value of a test: then you need to declare the `chisq_obj` and then call it as above, instead of using `chisq(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.chisq` is a string "chisq" , instead of a function.

Please see `CIT.py` for more details on the implementation of the (conditional) independent tests.

Parameters

`data` : `numpy.ndarray`, shape `(n_samples, n_features)`. Data, where `n_samples` is the number of samples and `n_features` is the number of features.

`method` : string, "chisq".

`kwargs` : e.g., `cache_path` . See Advanced Usages .

Returns

`p` : the p-value of the test.

Kernel-based conditional independence (KCI) test and independence test

Kernel-based conditional independence (KCI) test and independence test [1] . To test if `x` and `y` are conditionally or unconditionally independent on `Z`. For unconditional independence tests, `Z` is set to the empty set.

Usage

```
from causallearn.utils.cit import CIT
kci_obj = CIT ( data , "kci" ) # construct a CIT instance with data and method name
pValue = kci_obj ( X , Y , S )
```

The above code runs KCI with the default parameters. Or instead if you would like to specify some parameters of KCI, you may do it by e.g.,

```
kci_obj = CIT ( data , "kci" , kernelZ = 'Polynomial' , approx =
False , est_width = 'median' , ... )
```

See KCI.py for more details on the parameters options of the KCI tests.

Please be kindly informed that we have refactored the independence tests from functions to classes since the release v0.1.2.8 . Speed gain and a more flexible parameters specification are enabled.

For users, you may need to adjust your codes accordingly. Specifically, if you are

- running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

```
from causallearn.search.ConstraintBased.PC import pc from
causallearn.utils.cit import kci cg = pc ( data , 0.05 , kci )
```

- explicitly calculating the p-value of a test: then you need to declare the kci_obj and then call it as above, instead of using kci(data, X, Y, condition_set) as before. Note that now causallearn.utils.cit.kci is a string "kci" , instead of a function.

explicitly calculating the p-value of a test: then you need to declare the kci_obj and then call it as above, instead of using kci(data, X, Y, condition_set) as before. Note that now causallearn.utils.cit.kci is a string "kci" , instead of a function.

Please see CIT.py for more details on the implementation of the (conditional) independent tests.

Parameters

data : numpy.ndarray, shape (n_samples, n_features). Data, where n_samples is the number of samples and n_features is the number of features.

method : string, "kci".

kwargs :

- Either for specifying parameters of KCI, including: KernelX/Y/Z (condition_set) : ['Gaussian', 'Linear', 'Polynomial']. (For 'Polynomial', the default degree is 2. Currently, users can change it by setting the 'degree' of 'class PolynomialKernel()').

est_width : set kernel width for Gaussian kernels. 'empirical': set kernel width using empirical rules (default). 'median': set kernel width using the median trick. polyd : polynomial kernel degrees (default=2). kwidthx/y/z : kernel width for data x/y/z (standard deviation sigma). and more : see KCI.py for details.

Either for specifying parameters of KCI, including:

KernelX/Y/Z (condition_set) : ['Gaussian', 'Linear', 'Polynomial']. (For 'Polynomial', the default degree is 2. Currently, users can change it by setting the 'degree' of 'class PolynomialKernel()').

- 'empirical': set kernel width using empirical rules (default).

'empirical': set kernel width using empirical rules (default).

- 'median': set kernel width using the median trick.

'median': set kernel width using the median trick.

polyd : polynomial kernel degrees (default=2).

kwidthx/y/z : kernel width for data x/y/z (standard deviation sigma).

and more : see KCI.py for details.

- Or for advanced usages of CIT, e.g., cache_path . See Advanced Usages .

Or for advanced usages of CIT, e.g., cache_path . See Advanced Usages .

Returns

p : the p value.

Zhang, K., Peters, J., Janzing, D., & Schölkopf, B. (2011, July). Kernel-based Conditional Independence Test and Application in Causal Discovery. In 27th Conference on Uncertainty in Artificial Intelligence (UAI 2011) (pp. 804-813). AUAI Press.

G-Square test

Perform an independence test using G-Square test [1] . This test is based on the log likelihood ratio test.

Usage

```
from causallearn.utils.cit import CIT gsq_obj = CIT ( data ,  
"gsq" ) # construct a CIT instance with data and method name  
pValue = gsq_obj ( X , Y , S )
```

Please be kindly informed that we have refactored the independence tests from functions to classes since the release v0.1.2.8 . Speed gain and a more flexible parameters specification are enabled.

For users, you may need to adjust your codes accordingly. Specifically, if you are

- running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

running a constraint-based algorithm from end to end: then you don't need to change anything. Old codes are still compatible. For example,

```
from causallearn.search.ConstraintBased.PC import pc from  
causallearn.utils.cit import gsq cg = pc ( data , 0.05 , gsq )
```

- explicitly calculating the p-value of a test: then you need to declare the `gsq_obj` and then call it as above, instead of using `gsq(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.gsq` is a string "gsq" , instead of a function.

explicitly calculating the p-value of a test: then you need to declare the `gsq_obj` and then call it as above, instead of using `gsq(data, X, Y, condition_set)` as before. Note that now `causallearn.utils.cit.gsq` is a string "gsq" , instead of a function.

Please see `CIT.py` for more details on the implementation of the (conditional) independent tests.

Parameters

`data` : `numpy.ndarray`, shape (n_samples, n_features). Data, where `n_samples` is the number of samples and `n_features` is the number of features.

`method` : string, "gsq".

`kwargs` : e.g., `cache_path` . See Advanced Usages .

Returns

p: the p-value of the test

Tsamardinos, I., Brown, L. E., & Aliferis, C. F. (2006). The max-min hill-climbing Bayesian network structure learning algorithm. Machine learning, 65(1), 31-78.

Score functions

In this section, we would like to introduce score functions in causal-learn. Currently we have BIC score [1] , BDeu score [2] , generalized score with cross validation or marginal likelihood [3] .

Contents:

- BIC score Usage Parameters Returns
- Usage
- Parameters
- Returns
- BDeu score Usage Parameters Returns
- Usage
- Parameters
- Returns
- Generalized score with cross validation Generalized score with cross validation for single-dimensional variables Generalized score with cross validation for multi-dimensional variables
- Generalized score with cross validation for single-dimensional variables
- Generalized score with cross validation for multi-dimensional variables
- Generalized score with marginal likelihood Generalized score with marginal likelihood for single dimensional variables Generalized score with marginal likelihood for multi-dimensional variables
- Generalized score with marginal likelihood for single dimensional variables
- Generalized score with marginal likelihood for multi-dimensional variables

Schwarz, G. (1978). Estimating the dimension of a model. The annals of statistics, 461-464.

Buntine, W. (1991). Theory refinement on Bayesian networks. In Uncertainty proceedings 1991 (pp. 52-60). Morgan Kaufmann.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 1551-1560).

BIC score

Calculate the local score with Bayesian Information Criterion (BIC 
) for the linear Gaussian case.

Usage

```
from causallearn.score.LocalScoreFunction import local_score_BIC
score = local_score_BIC ( Data , i , PAi , parameters )
```

Parameters

Data : (sample, features).

i : current index.

PAi : parent indexes.

parameters : None.

Returns

score : Local BIC score.

Schwarz, G. (1978). Estimating the dimension of a model. The annals of statistics, 461-464.

BDeu score

Calculate the local score with BDeu 
) for the discrete case.

Usage

```
from causallearn.score.LocalScoreFunction import local_score_BDeu
score = local_score_BDeu ( Data , i , PAi , parameters )
```

Parameters

Data : (sample, features).

i : current index.

PAi : parent indexes.

- sample_prior: sample prior.

sample_prior: sample prior.

- structure_prior: structure prior.

structure_prior: structure prior.

- r_i_map: number of states of the finite random variable 'X_{i}'.

r_i_map: number of states of the finite random variable 'X_{i}'.

Returns

score : Local BDeu score.

Buntine, W. (1991). Theory refinement on Bayesian networks. In Uncertainty proceedings 1991 (pp. 52-60). Morgan Kaufmann.

Generalized score with cross validation

Generalized score with cross validation for single-dimensional variables

Calculate the local score using negative k-fold cross-validated log likelihood as the score, based on a regression model in RKHS [1] .

Usage

```
from causallearn.score.LocalScoreFunction import  
local_score_cv_general score = local_score_cv_general ( Data ,  
Xi , PAi , parameters )
```

Parameters

Data : (sample, features).

Xi : current index.

PAi : parent indexes.

- kfold: the fold number in cross validation.

kfold: the fold number in cross validation.

- lambda: regularization parameter.

lambda: regularization parameter.

Returns

score : Local score.

Generalized score with cross validation for multi-dimensional variables

Calculate the local score using negative k-fold cross-validated log likelihood as the score, based on a regression model in RKHS for data with multi-dimensional variables [1] .

Usage

```
from causallearn.score.LocalScoreFunction import  
local_score_cv_multi score = local_score_cv_multi ( Data , Xi ,  
PAi , parameters )
```

Parameters

Data : (sample, features).

Xi : current index.

PAi : parent indexes.

- kfold: the fold number in cross validation.

kfold: the fold number in cross validation.

- lambda: regularization parameter.

lambda: regularization parameter.

- dlabel: indicate the data dimensions that belong to each variable. It is only used when the variables have multivariate dimensions.

dlabel: indicate the data dimensions that belong to each variable. It is only used when the variables have multivariate dimensions.

Returns

score : Local score.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 1551-1560).

Generalized score with marginal likelihood

Generalized score with marginal likelihood for single dimensional variables

Calculate the local score by negative marginal likelihood, based on a regression model in RKHS [1] .

Usage

```
from causallearn.score.LocalScoreFunction import  
local_score_marginal_general score = local_score_marginal_general  
( Data , Xi , PAi , parameters )
```

Parameters

Data : (sample, features).

Xi : current index.

PA_i : parent indexes.

parameters : None.

Returns

score : Local score.

Generalized score with marginal likelihood for multi-dimensional variables

Calculate the local score by negative marginal likelihood, based on a regression model in RKHS for data with multi-dimensional variables [1] .

Usage

```
from causallearn.score.LocalScoreFunction import  
local_score_marginal_multi score = local_score_marginal_multi  
( Data ,  $X_i$  ,  $PA_i$  , parameters )
```

Parameters

Data : (sample, features).

X_i : current index.

PA_i : parent indexes.

- dlabel: indicate the data dimensions that belong to each variable. It is only used when the variables have multivariate dimensions.

dlabel: indicate the data dimensions that belong to each variable. It is only used when the variables have multivariate dimensions.

Returns

score : Local score.

Huang, B., Zhang, K., Lin, Y., Schölkopf, B., & Glymour, C. (2018, July). Generalized score functions for causal discovery. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 1551-1560).

Utilities

In this section, we would like to introduce utilities in causal-learn, such as graph operations and evaluations.

Contents:

- Graph operations DAG2CPDAG Usage Parameters Returns DAG2PAG Usage Parameters Returns PDAG2DAG Usage Parameters Returns TXT2GeneralGraph Usage Parameters Returns
 - DAG2CPDAG Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - DAG2PAG Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - PDAG2DAG Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - TXT2GeneralGraph Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - Evaluations Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - Datasets Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - Graph Visualization Usage Basic Plot Plot with Custom Node Labels Plot with Manual Node Colors Plot with Category-based Colors (Auto-assigned) Full Example: GES + Colored Visualization Parameters `get_category_colors()` Notes
 - Usage Basic Plot Plot with Custom Node Labels Plot with Manual Node Colors Plot with Category-based Colors (Auto-assigned) Full Example: GES + Colored Visualization
 - Basic Plot
 - Plot with Custom Node Labels
 - Plot with Manual Node Colors
 - Plot with Category-based Colors (Auto-assigned)
 - Full Example: GES + Colored Visualization
 - Parameters
 - `get_category_colors()`
 - Notes
-

Graph operations

In this section, we would like to introduce graph operations in causal-learn.

Contents:

- DAG2CPDAG Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - DAG2PAG Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - PDAG2DAG Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
 - TXT2GeneralGraph Usage Parameters Returns
 - Usage
 - Parameters
 - Returns
-

DAG2CPDAG

Convert a DAG to its corresponding CPDAG.

Usage

```
from causallearn.utils.DAG2CPDAG import dag2cpdag
CPDAG = dag2cpdag ( G )
```

Parameters

G : Direct Acyclic Graph.

Returns

CPDAG : Completed Partially Direct Acyclic Graph.

DAG2PAG

Convert a DAG to its corresponding PAG.

Usage

```
from causallearn.utils.DAG2PAG import dag2pag
PAG = dag2pag
( dag , islatent )
```

Parameters

dag : Direct Acyclic Graph.

islatent : the indexes of latent variables. [] means there is no latent variable.

Returns

PAG : Partial Ancestral Graph.

PDAG2DAG

Convert a PDAG to its corresponding DAG.

Usage

```
from causallearn.utils.PDAG2DAG import pdag2dag
Gd = pdag2dag
( G )
```

Parameters

G : Partially Direct Acyclic Graph.

Returns

Gd : Direct Acyclic Graph.

TXT2GeneralGraph

Convert text file of Tetrad results into GeneralGraph class for causal-learn.

Usage

```
from causallearn.utils.TXT2GeneralGraph import txt2generalgraph G
= txt2generalgraph ( filename )
```

Parameters

filename : Text file of Tetrad results.

Returns

G : GeneralGraph Class for causal-learn.

Evaluations

Usage

```
from causallearn.graph.ArrowConfusion import ArrowConfusion from
causallearn.graph.AdjacencyConfusion import AdjacencyConfusion
from causallearn.graph.SHD import SHD # For arrows arrow =
ArrowConfusion ( truth_cpdag , est ) arrowsTp = arrow .
get_arrows_tp () arrowsFp = arrow . get_arrows_fp () arrowsFn =
arrow . get_arrows_fn () arrowsTn = arrow . get_arrows_tn ()
arrowPrec = arrow . get_arrows_precision () arrowRec = arrow .
get_arrows_recall () # For adjacency matrices adj =
AdjacencyConfusion ( truth_cpdag , est ) adjTp = adj . get_adj_tp
() adjFp = adj . get_adj_fp () adjFn = adj . get_adj_fn () adjTn =
adj . get_adj_tn () adjPrec = adj . get_adj_precision () adjRec =
adj . get_adj_recall () # Structural Hamming Distance shd = SHD
( truth_cpdag , est ) . get_shd ()
```

Parameters

X : Data with T*D dimensions.

truth_cpdag : Graph class.

est : Graph class.

Returns

arrowsTp/Fp/Fn/Tn : True positive/false positive/false negative/true negative arrows.

arrowPrec : Precision for arrows.

arrowRec : Recall for arrows.

adjTp/Fp/Fn/Tn : True positive/false positive/false negative/true negative edges.

adjPrec : Precision for the adjacency matrix.

adjRec : Recall for the adjacency matrix.

shd : Structural Hamming Distance.

Datasets

Usage

```
from causallearn.utils.Dataset import load_dataset data , labels =  
load_dataset ( dataset_name )
```

Parameters

dataset_name : str, the name of a dataset in ['sachs', 'boston_housing', 'airfoil']

Returns

data : np.array, data with a shape of (number of samples, number of variables).

labels : list, labels of variables in the data.

Graph Visualization

GraphUtils.plot_graph() renders causal graphs with optional colored nodes. It supports manual per-node coloring, automatic category-based coloring, custom labels, titles, and saving to file.

Usage

Basic Plot

```
from causallearn.search.ScoreBased.GES import ges from
causallearn.utils.GraphUtils import GraphUtils # Run GES to get a
graph Record = ges ( X ) # Simple plot (no colors) GraphUtils .
plot_graph ( Record [ 'G' ] )
```

Plot with Custom Node Labels

```
GraphUtils . plot_graph ( Record [ 'G' ] , labels = [ 'Age' ,
'Income' , 'Spending' , 'Education' ] )
```

Plot with Manual Node Colors

```
# Assign a specific color to each node by name colors = { 'Age' :
'lightblue' , 'Income' : 'lightcoral' , 'Spending' :
'lightgreen' , 'Education' : 'lightsalmon' , } GraphUtils .
plot_graph ( Record [ 'G' ] , labels = [ 'Age' , 'Income' ,
'Spending' , 'Education' ] , colors = colors , title = 'Causal
Graph with Manual Colors' , save_path = 'my_graph.png' )
```

Plot with Category-based Colors (Auto-assigned)

```
# Group features into categories – each category gets a unique
color # Node labels are auto-derived from the category dict
(flattened in order) category_to_features = { 'demographics' :
[ 'Age' , 'Education' ] , 'financial' : [ 'Income' ,
'Spending' ] , } GraphUtils . plot_graph ( Record [ 'G' ] ,
category_to_features = category_to_features , title = 'Causal
Graph by Category' , save_path = 'graph_by_category.png' )
```

Full Example: GES + Colored Visualization

```
import numpy as np from causallearn.search.ScoreBased.GES import
ges from causallearn.utils.GraphUtils import GraphUtils # Generate
sample data np . random . seed ( 42 ) n = 500 X = np . random .
randn ( n , 5 ) X [:, 1 ] = X [:, 0 ] * 0.8 + np . random . randn
( n ) * 0.5 X [:, 2 ] = X [:, 1 ] * 0.6 + np . random . randn
```

```
( n ) * 0.4 # Run GES
Record = ges ( X , node_names = [ 'Drug_A' ,
'Drug_B' , 'Recovery' , 'Side_Effect' , 'Dosage' ]) # Visualize
with category colors
categories = { 'treatment' : [ 'Drug_A' ,
'Drug_B' , 'Dosage' ], 'outcome' : [ 'Recovery' ,
'Side_Effect' ], }
GraphUtils . plot_graph ( Record [ 'G' ],
category_to_features = categories , title = 'Drug Study Causal
Graph' , save_path = 'drug_study.png' , dpi = 300 )
```

Parameters

G : Graph. A causal-learn graph object (e.g., output of `ges()` or `dges()`).

labels : list of str or None. Node labels. Must have the same length as `G.get_nodes()` . Default: None (uses graph node names).

colors : dict or None. Mapping from node label (str) to fill color (str). Any valid CSS color name can be used (e.g., 'lightblue' , '#FF6B6B'). If both colors and `category_to_features` are provided, colors takes precedence. Default: None.

category_to_features : dict or None. Mapping from category name (str) to list of feature names (list of str). Each category is automatically assigned a distinct color from a built-in palette. If labels is not provided, node labels are auto-derived by flattening this dict in order. Default: None.

save_path : str or None. File path to save the rendered image (e.g., 'output/graph.png'). Supports PNG, PDF, SVG formats. If None, the image is only displayed. Default: None.

title : str. Title displayed above the graph. Default: "" .

dpi : float. Resolution in dots per inch. Default: 500.

figsize : tuple. Figure size in inches (width, height). Default: (20, 12).

get_category_colors()

A helper function that generates a color mapping from a category dictionary. Used internally by `plot_graph()` , but can also be called directly if you need to customize the color mapping before plotting.

```
from causallearn.utils.GraphUtils import GraphUtils
categories = { 'treatment' : [ 'Drug_A' , 'Drug_B' ], 'outcome' :
[ 'Recovery' , 'Side_Effect' ], }
colors = GraphUtils .
get_category_colors ( categories ) # Result: {'Drug_A':
'lightblue', 'Drug_B': 'lightblue', # 'Recovery':
```

```
'lightcoral', 'Side_Effect': 'lightcoral'} # Customize one color,
then plot colors [ 'Drug_A' ] = 'gold' GraphUtils . plot_graph
( G , colors = colors )
```

Parameters :

- `category_to_features` : dict. Mapping from category name to list of feature names.

`category_to_features` : dict. Mapping from category name to list of feature names.

Returns :

- `feature_to_color` : dict. Mapping from each feature name (str) to a CSS color string. All features in the same category share the same color.

`feature_to_color` : dict. Mapping from each feature name (str) to a CSS color string. All features in the same category share the same color.

Notes

- Dependencies : Requires matplotlib and pydot (which requires Graphviz to be installed).

Dependencies : Requires matplotlib and pydot (which requires Graphviz to be installed).

- Edge notation : Directed edges are shown as arrows. Undirected edges are shown as lines.

Edge notation : Directed edges are shown as arrows. Undirected edges are shown as lines.

- Color priority : If both colors and `category_to_features` are passed, colors is used and `category_to_features` is ignored.

Color priority : If both colors and `category_to_features` are passed, colors is used and `category_to_features` is ignored.

- Nodes without colors : If a node label is not found in the colors dict, it will be rendered with the default white background.

Nodes without colors : If a node label is not found in the colors dict, it will be rendered with the default white background.